

TEORIA DA COMPUTAÇÃO: RIGOR E CÓDIGO

PROF. DR. JEFFERSON O. ANDRADE

Created: 2026-03-19 qui 13:46

O DILEMA DA TEORIA

A VISÃO COMUM

- Teoria: Abstrata, antiga (anos 50), "inútil" na prática.
- Prática: Superficial, focada apenas em "fazer funcionar".

NOSSA PROPOSTA

- **União:** O rigor de Michael Sipser com a elegância de linguagens funcionais.
- **Mantra:** "Se você não consegue implementar, você não entendeu a teoria."

O KIT DE SOBREVIVÊNCIA

BIBLIOGRAFIA BASE

1. **Teoria Pura:** Sipser (Rigor matemático).
2. **Prática:** MacCormick (Intuição e aplicação).
3. **Ponte:** Pierce (Cálculo lambda e tipos).
4. **Modernidade:** Barak (Circuitos e complexidade).

POR QUE O PARADIGMA FUNCIONAL?

CORRESPONDÊNCIA DE CURRY-HOWARD

- Provas são programas.
- Tipos são proposições.

RECURSÃO E INDUÇÃO

- São a linguagem nativa da Teoria da Computação.

EXEMPLO: DFA EM HASKELL

```
type State = Int
type Alphabet = Char
type Transition = State -> Alphabet -> State

dfa :: Transition -> State -> [Alphabet] -> State
dfa delta q0 input = foldl delta q0 input
```

- Simples, puro e matematicamente idêntico à definição formal.

DINÂMICA DE AVALIAÇÃO

GRADUAÇÃO (FOCO: CONSTRUÇÃO)

- **40% Laboratórios Semanais:** Implementações curtas.
- **50% Projeto Final:** Uma ferramenta funcional completa.
- **10% Vídeo Demonstrativo:** 15-20 min de explicação técnica.

MESTRADO (FOCO: PESQUISA)

- **30% Prova de Rigor Teórico:** Provas e teoremas (Sipser).
- **70% Artigo Final:** Alvo: Submissão em congressos (SBC/IEEE).

O ROADMAP DO SEMESTRE

MÓDULO 1: LINGUAGENS REGULARES

- **Semanas 01-03:** Autômatos Finitos, NFA e Lema do Bombeamento.

MÓDULO 2: GRAMÁTICAS E PARSING

- **Semanas 04-06:** CFGs e o poder dos *Parser Combinators*.

MÓDULO 3: O LIMITE DA COMPUTAÇÃO

- Semanas 07-09: Máquinas de Turing e λ -cálculo.

MÓDULO 4: COMPLEXIDADE

- **Semanas 10-12: P vs NP e Redutibilidade.**

TRILHAS DE PESQUISA (MESTRADO)

1. AI SAFETY

- Verificação formal de agentes autônomos.

2. SMART CONTRACTS

- DSLs seguras para Blockchain.

3. GREEN COMPUTING

- Eficiência energética via Máquinas de Redução.

4. ZERO-KNOWLEDGE PROOFS

- Privacidade e complexidade computacional.

O ARTIGO FINAL

O QUE BUSCAMOS?

- **Originalidade:** Teoria aplicada a problemas de 2026.
- **Rigor:** Definições formais sólidas.
- **Reprodutibilidade:** Código funcional no GitHub.
- **Impacto:** Preparado para submissão imediata.

PRÓXIMOS PASSOS

ESCOLHA SUA FERRAMENTA

Você poderá optar por uma das seguintes linguagens funcionais:

- **Haskell:** O clássico purista.
- **F#:** Elegância funcional no ecossistema .NET.
- **Scala:** Poder e tipagem forte na JVM.
- **Clojure:** O pragmatismo dos Lisp modernos.

SETUP

1. Instalar o *toolchain* da linguagem escolhida.
2. **Lab 01:** Simulador de DFA (Entrega na próxima semana).

DÚVIDAS?

*"A computação é a primeira ciência
que nos permite criar as máquinas
que estudamos."*